

Traditional ML in RAG/MAS

Тема 4.3: традиционный ML в продвинутых задачах
Трек: advanced

Фирсанова В.И.

НИУ ВШЭ

Традиционный ML в эпоху LLM

- С ростом LLM интерес к традиционному ML не угас, а сместился в сторону:
 - **Оркестрации** для управление потоками запросов к LLM
 - **Препроцессинга** для подготовки данных перед векторизацией в RAG
 - **Постпроцессинга** для кластеризации, классификации, ранжирования результатов и даже проверки ответов LLM на безопасность
- ML-алгоритмы решают простые и незатратные **инженерные задачи**, которые LLM решают плохо или дорого, либо рискованно

Solution Example: LLM Agent with ML-Enabled Tools

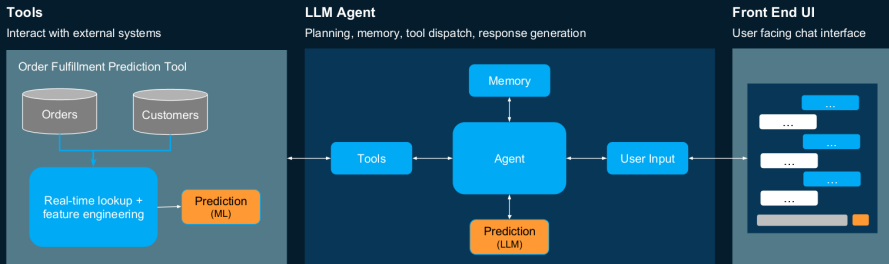


Рис.: Где можно применить традиционный ML в этой оркестровке?

Пример 1: Кластеризация в RAPTOR (RAG)

Проблема

Длинные документы (книги, исследования) не влезают в контекст LLM целиком.

Решение RAPTOR (Recursive Abstractive Processing for Tree-Organized Retrieval)

- 1 Разбить текст на чанки → эмбединги
- 2 Кластеризация (**Gaussian Mixture**) похожих чанков
- 3 LLM генерирует **суммаризацию** кластера
- 4 Повторять рекурсивно → строится дерево

ML-ядро: UMAP (снижение размерности) + Gaussian Mixture (кластеризация)

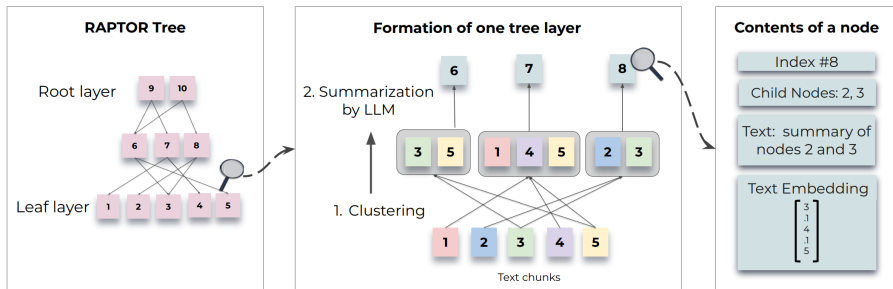


Рис.: Статья: RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval (arXiv:2401.18059)

Стратегии поиска

- **Tree Traversal** — спуск по дереву
- **Collapsed Tree** — поиск по всем узлам сразу (эффективнее)

Пример 2: Decision Tree как оркестратор LLM

Проблема

LLM деградирует при перегрузке контекста. Нужно подавать информацию порциями.

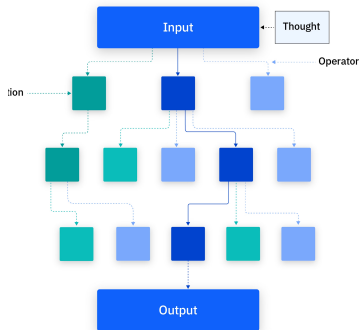
Решение: Prompt Tree — управление диалогом через дерево решений

```
class PromptBranch:
    def get_prompt(self) -> str:
        return prompt for this branch

    def get_tools(self) -> list[Tool]:
        return [switch_branch, update_insight]
```

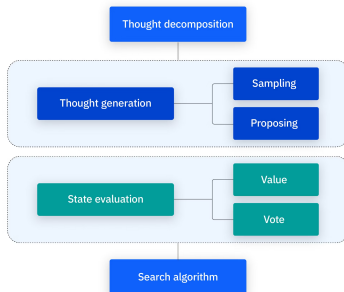
Применение: чат-боты со сложной логикой, анкетирование с ветвлением

Tree of Thoughts (ToT)



A single path in the tree yields the end solution

Рис.: For example, solving a sudoku puzzle might guide the model to explore different number placements in a trial-and-error fashion. It then backtracks when a number leads to a contradiction and it tries a different number until the puzzle is solved.



Thought decomposition breaks a problem into smaller steps (thoughts)

Sampling generates several thoughts independently by using the same prompt, while **Proposing** sequentially generates thoughts upon previous ones

Value strategy assigns a scalar value or a class to each state (likely/unlikely), while **Vote** strategy compares different solutions and selects the best.

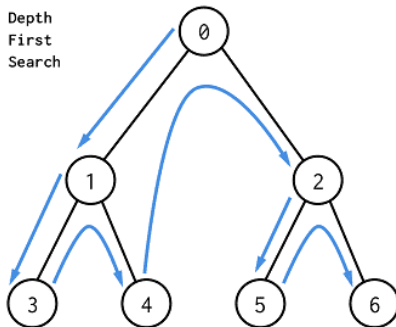
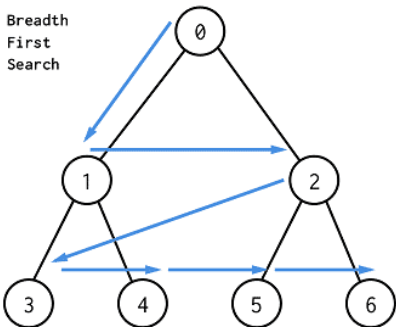


Рис.: Search algorithm is used to navigate through the solution space.

- **Breadth-first search (BFS)** explores all possible branches at each level before moving deeper into the tree.
- **Depth-first search (DFS)** explores one branch deeply before backtracking to explore other branches.

DAG-фищи в приложениях

The screenshot displays the Prompt Tree application interface, which uses a Directed Acyclic Graph (DAG) to manage a sequence of prompts and tool calls.

Left Panel (Library):

- Prompt Tree** (DIALOGUE TOPOLOGY)
- New Thread** (+)
- New Folder** (📁)
- LIBRARY**
- folder** (📁)
- FixNPM** (🕒 3 minutes ago 2 threads)
- Settings** (⚙️)

Main Panel (FixNPM):

- 6 nodes 2 branches**
- SETTINGS** (⚙️) **LIGHT** (☀️)
- Workflow Nodes:**
 - Node 1:** Prompt Tree, a assistant
 - Node 2:** How do I fix this npm issue (npm update)
 - Node 3:** Run npm audit fix to get a detailed breakdown of the...
 - Node 4:** Search for everything about npm
 - Node 5:** Complete Guide to npm (Node Package Manager)
 - Node 6:** Everything About npm The Complete Guide
- React Flow**
- Text:** Safest first step: Run `npm audit fix`. This will automatically update vulnerable packages in a non-breaking way (respects your `package.json` semver ranges).
- Input:** Type your message...
- Model:** grok-4-1-fast-reasoning, gln-4.7 - Temp: 0.7
- Models:** 2 **Tools:** 1 **Context:** 2,433 / 8,192

Right Panel (Context Assembly):

- BUILD YOUR PROMPT**
- TOKEN USAGE:** 2,433 / 8,192
- ACTIVE NODES:** Clear all
- TOOL BLOCKS:**
 - Tool Use: Web Search** (104)
 - You can search the web for up-to-date information. How to use: - Call 'web_search'..
 - System Prompt** (11)
 - You are Prompt Tree, a helpful assistant.
 - User Message** (118)
 - How do I fix this npm issue: npm update @yxp934/prompt-tree up to date, audited 533..
 - kfc · grok-4-1-fast-reasoning** (352)
- Compress Context (5)**
- Optimize Context**
- Preview Full Context** (>)

Рис.: <https://github.com/yxp934/Prompt-Tree>

DAG-фичи в приложениях

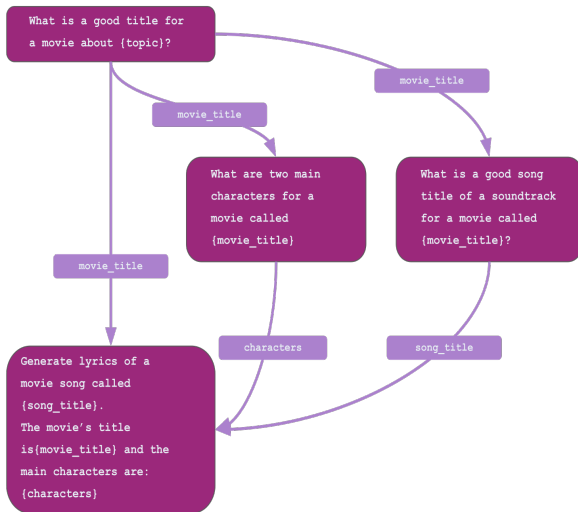


Рис.: LMFlow

Пример 3: Классификация для маршрутизации запросов

Проблема

Разные типы вопросов требуют разных стратегий обработки.

Решение: Классификатор (Logistic Regression / Random Forest) → маршрутизация

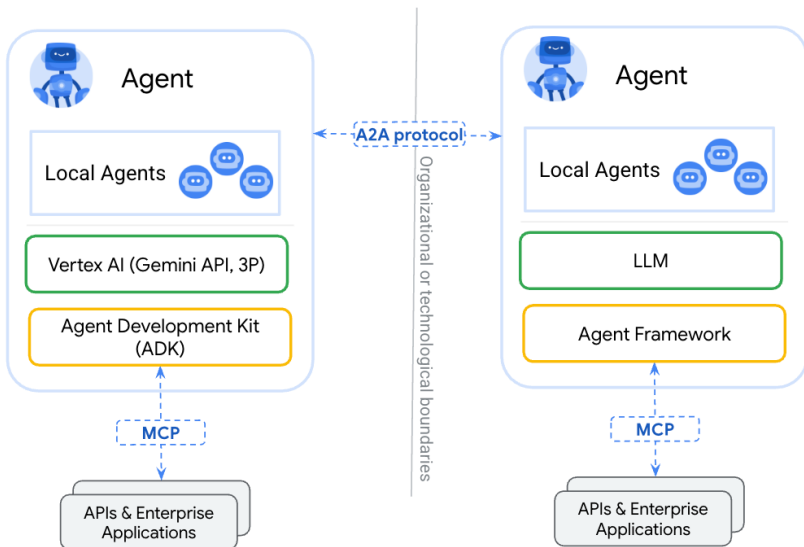
```
def route_query(query: str) -> str:
    features = extract_features(query)
    route = classifier.predict(features)

    if route == "math":
        return "calc_chain"
    elif route == "code":
        return "code_agent"
    else:
        return "default_rag"
```

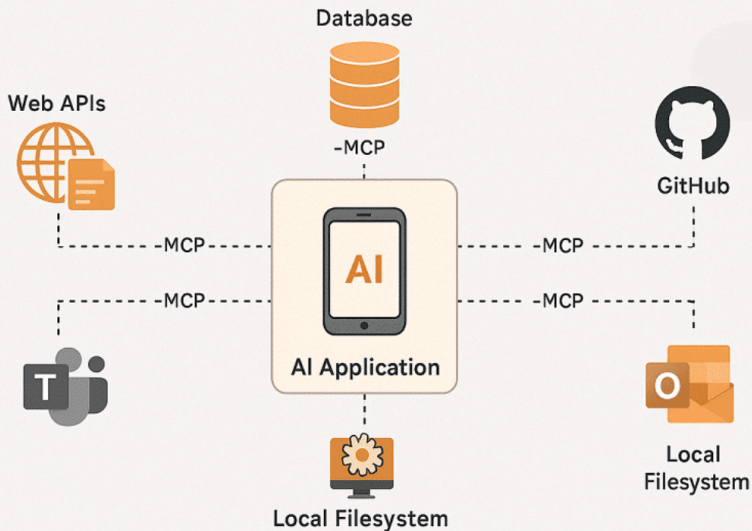
В библиотеках

Библиотека	Компонент	ML-алгоритм	Применение
LangChain	GraphTool	Knowledge Graph	RAG с графовой структурой
LangGraph	StateGraph	Граф состояний	Управление ветвлением
LlamaIndex	RAPTOR	GMM + UMAP	Иерархический поиск
CrewAI	Role-based agents	Неявно — tree of thought	Оркестрация агентов
Haystack	Pipeline DAG	Типизированные графы	Пайплайны

- **Протоколы:** MCP (Model Context Protocol), A2A (Agent-to-Agent)



What is MCP ?



Маршрутизация через ML (не LLM)

Архитектура

- ❶ Классификатор (Random Forest) определяет тип запроса
- ❷ В зависимости от типа — вызов нужной цепочки/агента

```
class Router:
```

```
    def __init__(self):
```

```
        self.clf = load_model("router.pkl")
```

```
        self.vectorizer = TfidfVectorizer()
```

```
    def route(self, query: str) -> str:
```

```
        X = self.vectorizer.transform([query])
```

```
        return self.clf.predict(X)[0]
```

- Latency <10 мс (vs LLM от 2-5 с до минуты)
- Бесплатно без API
- Децентрализация: один классификатор на одну задача

Ресурсы и ссылки

Статьи

- RAPTOR: [arXiv:2401.18059](https://arxiv.org/abs/2401.18059)
- LangChain Prompt Tree: [GitHub Issue #9932](#)

Библиотеки

- LangGraph — графовая оркестрация
- LlamaIndex — RAPTOR и продвинутая индексация
- CrewAI — мульти-агентные системы
- Haystack — production RAG

Сравнение фреймворков

morphllm.com/llm-frameworks